

Introdução ao OpenGL

Processamento de Imagens

Prof. Diógenes Furlan

Sumário

- GLUT
- OPENGGL
- ImageClass

Bibliotecas

- Padrão
 - Comando
 - `include <GL/gl.h>`
- GLU
 - OpenGL Utility Library
 - Comando
 - `include <GL/glu.h>`
- GLUT
 - OpenGL Utility Toolkit
 - independente da plataforma
 - Comando
 - `include <GL/glut.h>`

OPENGL

OpenGL

- Biblioteca de rotinas gráficas e de modelagem
 - bidimensional (2D)
 - tridimensional (3D)
- Portabilidade
- Rapidez
 - algoritmos cuidadosamente desenvolvidos e otimizados
 - Silicon Graphics, Inc.
 - líder mundial em Computação Gráfica e Animação.

Máquina de Estados

- OpenGL é uma máquina de estados, conforme descrito por [Woo 1999].
- É possível colocá-la em vários estados (ou modos) que não são alterados, a menos que uma função seja chamada para isto.
- Por exemplo, a cor corrente é uma variável de estado que pode ser definida como branco.
 - Todos os objetos, então, são desenhados com a cor branca, até o momento em que outra cor corrente é especificada.

Máquina de Estado

- Variáveis de Estado
 - Cor
 - Padrões de linhas
 - Características das luzes
 - Propriedades dos materiais
- Funções relacionadas
 - *glEnable()*
 - *glDisable()*

Convenção dos Comandos OpenGL

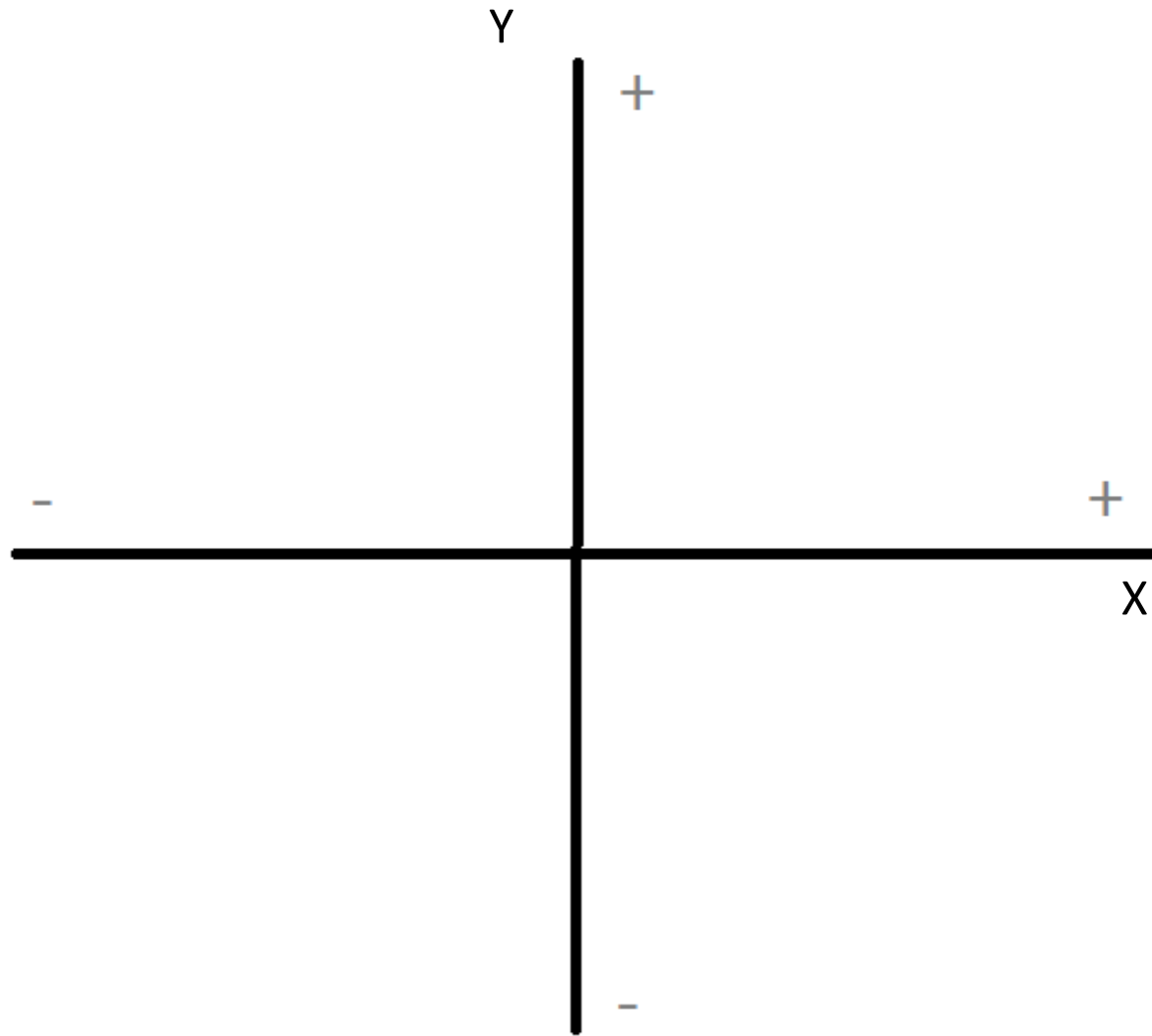
<A> <C> <D>

- ***A: <PrefixoBiblioteca>***
- ***B: <ComandoRaiz>***
- ***C: <ContadorArgumentosOpcional>***
- ***D: <TipoArgumentosOpcional>***
- Prefixos de biblioteca:
 - gl: OpenGL
 - glu: Glu
 - glut: Glut

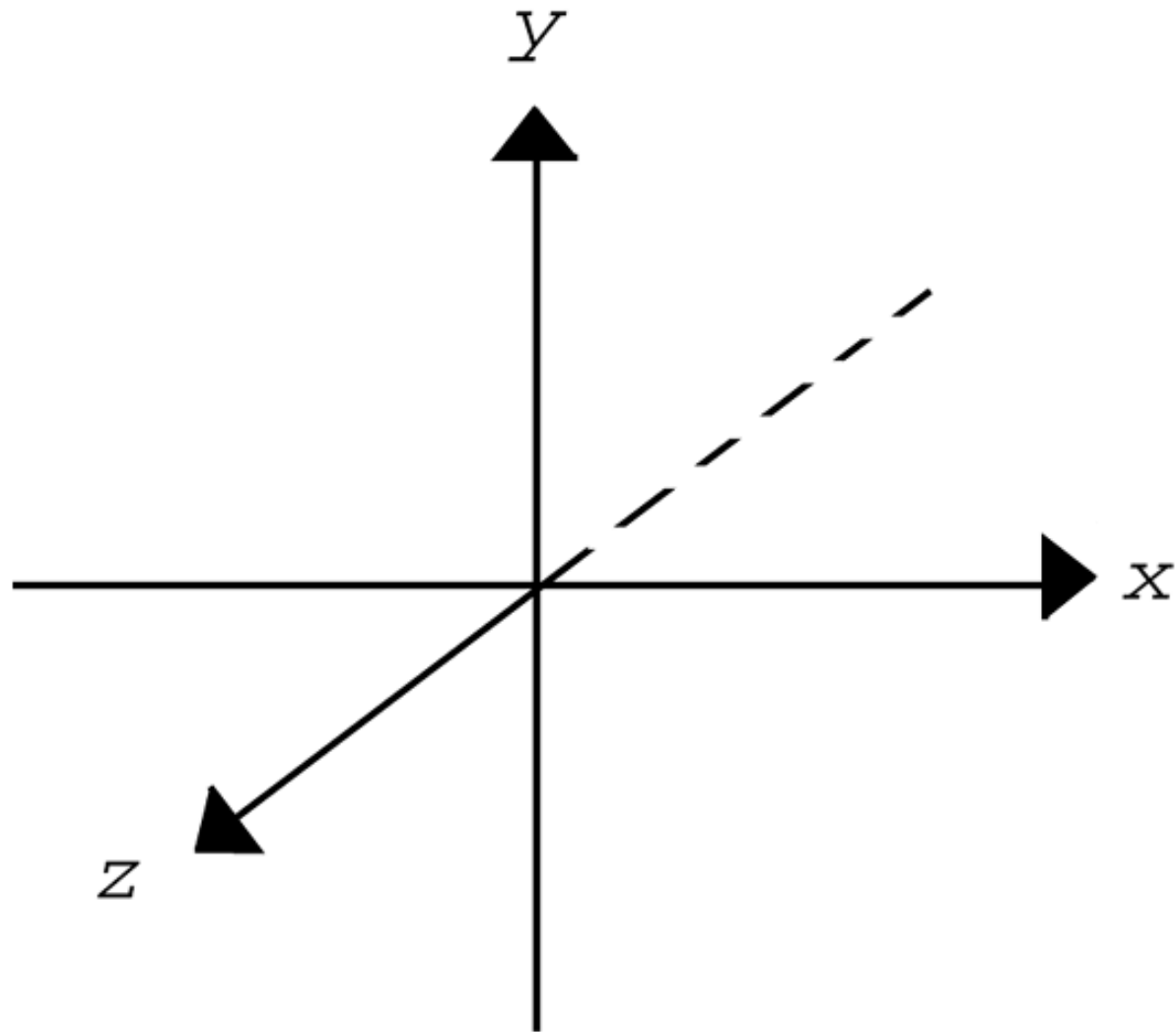
Exemplo: Sufixo das Funções de Cores

- Por exemplo, a função ***glColor3f***
 - *prefixo da biblioteca: gl*
 - *raiz: Color.*
 - *sufixo: 3f*
 - função possui três valores de ponto flutuante como parâmetro.
- Tipos:
 - *f: float*
 - *d: double*
 - *i: int*
- Valores:
 - *3 ou 4*

Quadrantes 2D no OpenGL



Quadrantes 3D no OpenGL



GLUT

GLUT

- É uma API que possibilita a codificação de programas em OpenGL, **independente da plataforma** de hardware.
- Desta forma um programa escrito utilizando-se OpenGL e GLUT pode potencialmente ser portado e compilado em plataformas diferentes de hardware, como estações de trabalho UNIX, Videogames e PCs.
- Foi desenvolvido por Mark J. Kilgard (Silicon Graphics)
 - inicialmente para uso em estações de trabalho UNIX.

Funções de Inicialização

- Inicializa o Glut
 - `glutInit()`
- Cria janela
 - `glutCreateWindow( titulo )`
- Define posição da janela
 - `glutInitWindowPosition( x, y )`
- Define dimensões da janela
 - `glutInitWindowSize( w, h )`
- Define propriedades gráficas
 - `glutInitDisplayMode( modo )`
- Loop principal
 - coloca o programa num loop infinito, esperando por eventos
 - `glutMainLoop()`

Cadastro de Funções de Call Back

- Desenho
 - `glutDisplayFunc( displayFunc )`
- Alterações na janela
 - `glutReshapeFunc( reshapeFunc )`
- Tempo ocioso
 - `glutIdleFunc( idleFunc )`
- Timer
 - `glutTimerFunc( timerFunc )`

Cadastro de Funções de Call Back

- Teclado
 - `glutKeyboardFunc( keyboardFunc )`
- Teclas especiais
 - `glutSpecialFunc( arrowKeysFunc )`
- Mouse
 - `glutMouseFunc( mouseFunc )`
- Joystick
 - `glutJoystickFunc( joystickFunc )`
- Menu
 - `glutMenuStatusFunc( menuFunc )`

Download

- Arquivo PDI-Aula-Aluno.zip
- Abrir o projeto no Code Blocks
- Menu Build / Rebuild
 - Reconstrói os arquivos na sua máquina
- Arquivo Imagens.zip
 - Arrumar a pasta (linha 100 do código)

Main

```
int main ( int argc, char** argv )
{
    glutInit                ( &argc, argv );
    glutInitDisplayMode (GLUT_DOUBLE | GLUT_DEPTH | GLUT_RGB );
    glutInitWindowPosition (100,100);

    // Define o tamanho da janela gráfica do programa
    glutInitWindowSize (800, 400);
    glutCreateWindow ( "Image Loader" );

    init ();
    glutFullScreen();

    glutDisplayFunc ( display );
    glutReshapeFunc ( reshape );
    glutKeyboardFunc ( keyboard );
    glutSpecialFunc ( arrow_keys );

    glutMainLoop ( );
}
```

Init

```
void init(void)
{
    int r;

    glClearColor(0.0f, 0.0f, 1.0f, 1.0f); // Fundo de tela azul

    Image = new ImageClass();
    if( Image->Load("../ImagensGL/abbey.bmp") )
        printf ("Imagem carregada!\n");
    else {
        printf("Imagem não encontrada.");
        exit(1);
    }

    NovaImagem = new ImageClass(Image->SizeX(), Image->SizeY());
    NovaImagem->setPos(Image->SizeX()+10, 0);
}
```

Display

```
void display( void )
{
    glClear(GL_COLOR_BUFFER_BIT);

    glMatrixMode(GL_MODELVIEW);
    Image->Display();
    NovaImage->Display();

    glutSwapBuffers();
}
```

Reshape

```
void reshape( int w, int h )
{
    // Reset the coordinate system before modifying
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    // Set the viewport to be the entire window
    glViewport(0, 0, w, h);
    gluOrtho2D(0,w,0,h);

    // Set the clipping volume
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}
```

Keyboard

```
void keyboard ( unsigned char key, int x, int y )
{
    switch ( key )
    {
        case 27:          // Termina o programa qdo
            exit ( 0 );    // a tecla ESC for pressionada
            break;
        case '1':
            CopiaImagem();
            glutPostRedisplay(); // obrigatório para redesenhar a tela
            break;
        case '2':
            ConvertToGrayScale();
            glutPostRedisplay(); // obrigatório para redesenhar a tela
            break;
        default:
            break;
    }
}
```

Arrow Keys

```
void arrow_keys ( int a_keys, int x, int y )
{
    switch ( a_keys )
    {
        case GLUT_KEY_UP:
            glutFullScreen ( );
            break;
        case GLUT_KEY_DOWN:
            glutInitWindowSize ( 700, 500 );
            break;
        default:
            break;
    }
}
```

EXERCÍCIOS

Exercícios

1. Modifique a cor de fundo da janela para verde.
2. Navegar por várias (ou todas) imagens .bmp do diretório usando as teclas de movimentação ($\leftarrow$, $\rightarrow$).
3. No algoritmo do **ConvertBlackAndWhite** controlar o limiar pelo teclado, usando duas teclas, uma para decrementar e outra para incrementar, de 5 em 5. Mostrar o valor do limiar atual na tela.